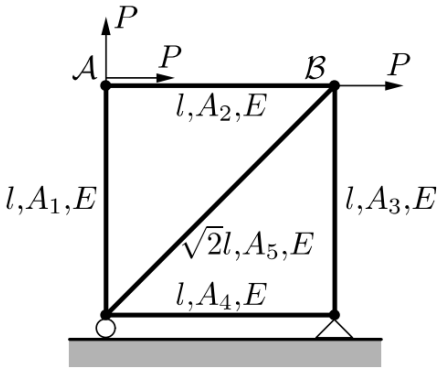
Numerical solution of a simple structural optimization problem

Training 4: CONLIN approximation

Text of the exercise

The mass of the five-bar truss in the figure below should be **minimized** given that the truss has to be sufficiently stiff. Specifically, the compliance has to be lower than the value C_0. All bars have Young's modulus E. The design variables are the cross-sectional areas of the bars: A_1, A_2, A_3, A_4, A_5.



Numerical data

Consider the following numerical data:

Density of the material, $\rho = 7'850\ \frac{\mathrm{kg}}{\mathrm{m}^3}$.

Length of bars 1, 2, 3 and 4, $l = 1.25\ \mathrm{m}$.

Elastic modulus of the material, $E = 210\mathrm{E}9\ \mathrm{Pa}$.

External load, $P = 5'000\ \mathrm{N}$.

Allowed maximum compliance, $C_0 = 0.5\ \mathrm{Nm}$.

Requests

1. **Solve** the problem **numerically**, using the CONLIN approximation;
2. Compare the **numerical** and the **analytical** (symbolic) results;
3. **How many iterations** are required to obtain a solution which matches to the symbolic one? Why?

Optimization problem formulation

The optimization problem was formulated during the last training lecture. Note that the objective function (mass of the structure) is a linear function of the five design variables, A_i. The compliance, instead, is a linear combination of their reciprocals, $\frac{1}{A_i}$.

$$\begin{cases} \min \ A_1 + A_2 + A_3 + A_4 + \sqrt{2}A_5 \\ \text{s.t.} \quad \begin{cases} \dfrac{1}{A_1} + \dfrac{1}{A_2} + \dfrac{4}{A_3} + \dfrac{4}{A_4} + \dfrac{8\sqrt{2}}{A_5} - \dfrac{Ec_0}{P^2 l} \leq 0 \\ A_1, \ldots, A_5 \geq 0. \end{cases} \end{cases}$$

In this exercise, there are no limits on the maximum value the cross sectional areas can have. The only constraint is they must be positive.

CONLIN approximation and solution of the problem

The problem is solved iteratively. That is, all the five design varibles are initialized to a value, then the CONLIN approximation is calculated around the guess and the resulting linearized problem is solved. The solution is used to calculate a new CONLIN approximation and linearized problem, which, once solved, returns a new set of values for the design variables. The iterative procedure stops when at least one of the User defined stopping criteria is met. We will set just **one stopping criterium**, which is a **maximum number of iterations** (i.e. we will iterate the procedure for a defined number of times).

CONLIN approximation

The objective function is already linear:

$$f = \rho \, l \sum_{j=1}^{5} a_j A_j$$

With $a_j = \left[1, 1, 1, 1, \sqrt{2}\right]^T$.

The constraint function $C - C_0 \leq 0$ is a linear combination of the reciprocal of the design variables, therefore it is linearized always using the reciprocal variables.

$$g = r_0 + \sum_{j=1}^{5} \frac{q_{0j}}{A_j} - C_0$$

With $r_0 = g(\boldsymbol{x}_0) + \sum_{j=1}^{5} \frac{\partial g}{\partial x_j} x_{0j}$ and $q_{0j} = -\frac{\partial g}{\partial x_j} x_{0j}^2$. Where $\boldsymbol{x}_0$ is the vector of the current values of the five design variables.

The Lagrangian of the linearized problem becomes:

$$L = \rho\, l \sum_{j=1}^{5} a_j A_j + \lambda\,(r_0 - C_0) + \lambda \sum_{j=1}^{5} \frac{q_{0j}}{A_j}$$

Thanks to the linearization, the **Lagrangian** is guaranteed to be a **separable** function, such that the partial derivative with respect to x_j becomes:

$$\frac{\partial L_j}{\partial A_j} = \rho\, l\, a_j \; - \lambda\, \frac{q_{0j}}{A_j^2}, \quad \text{with } j = 1, 2, 3, 4, 5.$$

Then, by solving $\frac{\partial L}{\partial A_j} = 0$ it is possible to obtain the **design variable expressed as function of the Lagrangian multiplier**:

$$A_j^*(\lambda) = \sqrt{\frac{\lambda\, q_{0j}}{\rho\, l\, a_j}}$$

The dual function $\varphi(\lambda)$ is:

$$\varphi(\lambda) = \rho\, l \sum_{j=1}^{5} a_j \sqrt{\frac{\lambda\, q_{0j}}{\rho\, l\, a_j}} + \lambda\,(r_0 - C_0) + \lambda \sum_{j=1}^{5} q_{0j} \sqrt{\frac{\rho\, l\, a_j}{\lambda\, q_{0j}}}$$

In order to solve the original problem, the maximum of the dual function must be found. Denoting with k the k-th iteration, the final problem becomes:

$$\begin{cases} \max\limits_{\lambda} \varphi^k(\lambda) \\ \text{s.t.} \quad \lambda \geq 0. \end{cases}$$

Where, if present, the bounds on the design variables (box constraints) must be kept into account.

It is usefult, in the numerical implementation of the solution algorithm, to keep into consideration that, for active constraints, this property holds:

$$\frac{d\varphi}{d\lambda}(\lambda) = g(\boldsymbol{x}^*(\lambda))$$

Note that at each iteration of the solution algorithm none of the equations written so far ceases to be valid. The only parameters which vary are just the two constants q_{0j} and r_0.

Matlab code

Input the numerical values into Matlab Workspace. When inserting data, always check the **consistency** of **units of measurement**.

```
% numerical values
rho = ...
L = ...
E = ...
P = ...
C0 = ...
```

Now, you should define the **number of iterations** the algorithm should perform. Create also arrays to store the solution of each iteration.

```
% number of iterations
N = ...
% arrays
x_sol = ... % array to save the solution
lambda = ... % vector to save the multiplier
```

Set up the iterative cycle. In order to numerically solve the problem, you should **initialize the** five **design variables** and the **Lagrangian multiplier**.

```
% initial guess
x0 = ... % initial guess for the five D.V.
l0 = ... % initial guess for the multiplier
% iterative cycle
for ii = 1 : N
    % sensitivity analysis
    grad_fun = [...
    % solve the function of lambda only using fsolve
    % dphi is a User-defined function that you have to write at the end of
    % your script
    lambda(ii) = fsolve(@(l) dphi(l, grad_fun, rho, E, l, C0, P), l0);
    % evaluate the new values of D.V. using the updated value of the
    % multiplier
    x_sol(:, ii) = ...
    % update the current variables initialization
    x0 = x_sol(:, ii);
    l0 = lambda(ii);
end
```

Type *help fsolve* into the Command Window to get instructions on how to use *fsolve*.

`X = fsolve(FUN,X0) starts at the matrix X0 and tries to solve the equations in FUN. FUN accepts input X and returns a vector (matrix) of equation values F evaluated at X.`

To see how the solution evolves through iterations, plot the value of the **mass of the structure** and the **value of the constraint**.

```
figure(1) % constraint
plot(...
%
figure(2) % mass
mass = ...
plot(...
```

Dual function

Define a Matlab function which returns the value y of the derivative of the dual function. Recall that:

$$y = \frac{d\varphi}{d\lambda}(\lambda) = g(\boldsymbol{x}^*(\lambda))$$

Where $\boldsymbol{x}^* = \left[A_1^*,\ A_2^*,\ A_3^*,\ A_4^*,\ A_5^*\right]^T$ and $A_j^*(\lambda) = \sqrt{\frac{\lambda\, q_{0j}}{\rho\, L\, a_j}}$.

```
function [y, x_new] = dphi(l, x, grad_fun, rho, E, L, C0, P)
    % l = Lagrangian multiplier (function variable)
    % x = set of D.V. values (column vector)
    % lb = lower bounds for the D.V. (column vector)
    % ub = upper bounds for the D.V. (column vector)
    % rho, E, L, C0, P = numerical values (properties of the problem)
    %
    % impose consistency for the multiplier (positive real number)
    if l < 0
        l = 0;
    end
    % constants q0 of the CONLIN approximation
    q0 = -grad_fun...
    % constants rho*L*a of the CONLIN approximation
    a = rho*...
    % compute x_new (function of lambda)
    x_new = sqrt(l)*...
    % derivative of the dual function
    y = ... % evaluate the linearized constraint at x*(l)
end
```